

Pulse App Improvement Audit

Repository: Maximilien-Mnt/Pulse

Revision reviewed: eeaf0b5 (main)

Review date: 2026-09-07

Stack: Expo 54, React Native 0.81, Expo Router, TypeScript strict mode, NativeWind, Supabase, TanStack Query, Zustand, PostHog.

Executive summary

Pulse already has a broad product surface: authentication and onboarding, multilingual UI, feed/posts/comments/likes, clubs, events, discovery, profiles, notifications, invitations, messaging, reporting, blocking, media, and legal pages. The strongest immediate opportunity is not adding more screens; it is making the existing product safer and more predictable. The repository currently exports successfully for web, but its baseline is not clean: TypeScript reports four concrete errors, two of four Jest suites fail, lint cannot run because `eslint` is not available through the declared project setup, Expo dependency alignment is stale, and the production web bundle is 6.48 MB of JavaScript.

The second major opportunity is consistency. Several repeated UI/data implementations—especially favorite buttons and counts on cards—perform separate Supabase queries per rendered item. Large screens such as the public club page, profile editing, dashboard, feed, and explore are monolithic, which makes low-cost IDE agents more likely to introduce regressions. Accessibility is partially present, but not systematized: some important controls have labels while many icon-only actions, images, dialogs, lists, and loading/error states do not appear to have a uniform contract. The recommended strategy is to fix correctness and observability first, then consolidate shared primitives and query patterns, then improve accessibility/performance, and only afterward add product features.

Evidence and scope

Commands and observations

Check	Result	Meaning
<code>npx expo export --platform web</code>	Passed	The web bundle can be produced at the reviewed revision.
Web bundle	6.48 MB JS, 31.5 kB CSS	High initial-load risk, especially on mobile/low-end devices.

TypeScript	Failed	Four reported error groups: conversation action-sheet props, unsupported <code>PenLine</code> icon, <code>ShareButton</code> missing <code>hitSlop</code> , and missing <code>club_id</code> in conversation mapping.
Jest	Failed: 2 suites, 1 test	<code>reset-password</code> suite fails because <code>t</code> is not mocked/available; forgot-password assertion expects shorter text than the rendered localized copy. Two signup-related suites pass.
Lint	Could not run	<code>npm run lint</code> resolves to <code>eslint</code> , but <code>eslint</code> is not installed in the declared dependencies/environment. Add a deliberate lint toolchain before treating lint as a gate.
Translation parity	Passed	<code>check_parity.cjs</code> : 1000 FR keys, 1000 EN keys, no duplicates or missing keys.
Dependency audit	Failed	<code>npm audit --omit=dev</code> : 36 findings (22 moderate, 14 high) in the current installed graph. Do not blindly run <code>npm audit fix -force</code> ; it proposes a breaking Expo upgrade.
Expo compatibility	Failed	<code>npx expo install --check</code> reports drift for Expo, Expo constants/file system, FlashList, keyboard controller, video thumbnails, and Jest Expo.
Visual browser review	Not completed	The connected browser was unavailable when opening localhost. Source-level review and web export evidence are included; perform device/browser verification before merging UI changes.

Important evidence distinction

Items marked **Confirmed** are directly supported by repository inspection or command output. Items marked **Recommended** are best-practice improvements that should be validated with profiling, accessibility tooling, or product analytics before implementation.

Prioritized implementation plan

P0 — Fix before adding features

1. Restore a green typecheck

Status: Confirmed.

Why: TypeScript failures hide real runtime/API drift and make cheap IDE agents unsafe: they may build on already-invalid assumptions. The reported failures are localized and should be fixed before refactoring.

What it should look like in the app: No deliberate visual change. Conversation menus open with the intended props, supported icons render, share controls accept touch props, and conversation list rows contain the complete typed shape.

Cline-ready prompt:

Plain Text

You are working in the Pulse Expo/React Native TypeScript repository.

Goal: make ``npx tsc --noEmit`` pass without changing product behavior.

Work only on the currently reported type errors:

1. `app/(tabs)/conversations/index.tsx` passes ``group``, but `ConversationActionSheet`
2. `components/conversations/ConversationActionSheet.tsx` uses the Icon name ``Pen``
3. `components/shared/ShareButton.tsx` passes ``hitSlop``, but the component props
4. `hooks/useConversations.ts` maps a row without the required ``club_id``. Select/

Steps:

- Read the affected files and existing types first.
- Make the smallest coherent changes.
- Run ``npx tsc --noEmit``.
- If a new error appears, fix only errors caused by this change.
- Report changed files, the exact validation command, and its result.
- Do not reformat unrelated files or upgrade dependencies.

2. Repair and strengthen the failing tests

Status: Confirmed.

Why: Two failing suites mean authentication regressions can ship unnoticed. The `t` failure also indicates a fragile module-level translation dependency in validation code.

What it should look like in the app: Password reset validation loads in tests and production; the forgot-password success state has one stable, translated message and tests assert the intended full copy without brittle accidental truncation.

Cline-ready prompt:

Plain Text

You are working in the Pulse repository. Make the Jest suite green while preserv

Failures to fix:

- `__tests__/reset-password.test.tsx`: `utils/validation.ts` calls ``t(...)`` at modu
- `__tests__/forgot-password.test.tsx`: the test expects a shorter string while t

First inspect:

- `utils/validation.ts`
- `lib/translations.ts` and `hooks/useTranslation.ts`
- `app/auth/reset-password.tsx`
- `app/auth/forgot-password.tsx`
- the affected tests and Jest setup

Preferred design:

- Avoid freezing language-dependent validation messages at module import time.
- Make the success message a translation key. Update the test to assert the int
- Add or update focused tests for English/French language switching if the exis

Validation:

- ``npm test -- --runInBand``
- ``npx tsc --noEmit``

Do not upgrade packages, rewrite unrelated tests, or use ``as any``.

3. Make the quality gates reproducible

Status: Confirmed.

Why: `npm run lint` cannot run because `eslint` is absent, and `expo-doctor` is not a declared project command. A green local export is not enough for a team project.

What it should look like in the app/project: A fresh clone can run install, typecheck, lint, tests, export, and diagnostics with documented commands. CI reports failures clearly.

Cline-ready prompt:

Plain Text

5. Fix optimistic favorite mutations

Status: Confirmed risk in source.

Why: Club and event card mutations always set favorite state to `true` and increment the count in `onMutate`, even when the user is un-favoriting. The subsequent refetch may correct it, but the immediate UI can show the wrong state/count and rapid taps can race.

What it should look like in the app: Tapping favorite immediately toggles the icon and count in the correct direction; failures roll back; duplicate taps are disabled while pending; the final server state is authoritative.

Cline-ready prompt:

Plain Text

Fix optimistic favorite toggling for clubs and events without changing the data

Inspect:

- `components/clubs/ClubCard.tsx`
- `components/clubs/ClubCardGrid.tsx`
- `components/events/EventCard.tsx`
- `components/events/EventCardGrid.tsx`
- the detail screens and any shared `FavoriteButton`

Requirements:

- Derive `nextIsFavorite` from the cached current boolean, not from an assumption
- Update the count by +1 for add and -1 for remove, never below zero.
- Cancel matching queries before the optimistic update.
- Snapshot the previous values and roll back on error.
- Invalidate/refetch the matching detail/list queries after settlement.
- Prevent duplicate mutation calls while pending, while retaining an accessible
- Keep query keys consistent and include user identity where the result is user
- Do not use `any` or duplicate the same algorithm four times if a small shared

Add focused tests for add, remove, and rollback. Run typecheck and Jest.

P1 — High-value architecture, accessibility, and performance work

6. Remove per-card Supabase query multiplication

Status: Confirmed pattern.

Why: Each rendered club/event card can issue a user-favorite query plus a count query, and cards repeat the implementation. A feed of 20 cards can create dozens of requests and independent cache entries. This hurts latency, battery, rate limits, and perceived speed.

What it should look like in the app: Lists render from one list query (or a small batched query) with favorite state/counts already attached; scrolling does not trigger a request per card.

Cline-ready prompt:

Plain Text

Reduce network request multiplication in club/event/feed cards.

First map all card-level Supabase queries for favorite state and counts. Do not

Implement a staged solution:

1. Create typed query helpers/hooks for batched favorite IDs for the current user.
2. Add the data at the list/container level and pass plain props into presentation.
3. Keep detail screens correct with their own query when needed.
4. Ensure query keys include entity type and user ID where required.
5. Preserve loading, error, empty, and optimistic states.
6. Avoid an N+1 client loop (`Promise.all` with one query per ID is not an acceptable pattern).
7. Add a lightweight test or instrumentation assertion that a list render does not trigger multiple queries.

Work on one entity type first, validate it, then apply the same pattern to the others.

7. Introduce pagination/infinite loading for feed, explore, comments, members, and messages

Status: Confirmed risk: several hooks/screens select broad datasets and do not expose a consistent infinite-query contract.

Why: Unbounded lists increase transfer size, memory, render cost, and database load as the app grows.

What it should look like in the app: Initial content appears quickly, a footer spinner loads the next page, pull-to-refresh resets to page one, and end-of-list is explicit. Comments/messages remain usable with large histories.

Cline-ready prompt:

Plain Text

Add cursor- or range-based pagination to the highest-volume Pulse lists, starting with

Inspect the current schema, ordering columns, hooks, list components, and Supabase queries.

Implement only the feed in this task:

- select explicit columns instead of `\*`;
- fetch a bounded page (for example 20–30, chosen from existing UX);

- expose `pages.flatMap` to the list;
- add `onEndReached` with a guard for loading/end states;
- preserve pull-to-refresh and invalidate behavior after create/delete/like/comment;
- show loading footer, empty state, and retry state;
- prevent duplicate pages and handle deleted rows safely;
- write a focused hook test for page progression and refresh reset.

Do not change ordering semantics or add virtualization until you inspect the current

8. Split monolithic screens and extract feature boundaries

Status: Confirmed maintainability issue.

Why: The public club page is about 1,122 lines; profile edit is about 855; club dashboard about 840; feed/explore are about 600+ each. Large files mix fetching, mutations, layout, formatting, modals, and platform behavior. Cheap models are more reliable with small files and explicit contracts.

What it should look like in the app: No intended visual change; behavior is divided into typed hooks, sections, and presentational components that can be tested independently.

Cline-ready prompt:

Plain Text

Refactor only ``app/(tabs)/clubs/[clubId]/index.tsx`` into smaller typed modules

Before editing, identify these boundaries in the existing file:

- data queries and derived state;
- header/hero;
- club metadata;
- membership/join actions;
- upcoming events;
- past events;
- gallery/media;
- favorite/share actions;
- loading/error/empty states;
- modals/sheets.

Extract components into ``components/clubs/public/`` and query/mutation logic into

Constraints:

- no behavior changes;
- no dependency changes;
- no broad formatting pass;
- no ``any``, ``as unknown as``, or suppressed diagnostics;
- each extracted component must have a typed props interface;
- run snapshot/targeted tests if available, typecheck, and web export.

Stop after this screen. Do not refactor the rest of the app in the same task.

Repeat the same one-screen task later for profile edit, dashboard, feed, and explore.

9. Establish an accessibility contract for all interactive primitives

Status: Recommended with partial gaps observed.

Why: Some screens already use accessibility roles/labels, but coverage is inconsistent. Icon-only favorite/share/delete/menu controls and modal states need predictable names, hints, state, and disabled/busy semantics.

What it should look like in the app: VoiceOver/TalkBack users can identify every control, know whether a favorite is active, understand validation errors, navigate dialogs, and reach all actions with a 44×44 logical touch target.

Cline-ready prompt:

Plain Text

Create and apply an accessibility contract for Pulse's shared UI primitives with

Inspect components/ui/Button.tsx, Input.tsx, PressableRow.tsx, PressableScale.t

Implement:

- typed `accessibilityLabel`, `accessibilityHint`, `accessibilityState`, and `a
- a consistent 44x44 minimum hit area for icon-only actions, using padding/hits
- disabled and pending states exposed to assistive technology;
- meaningful labels for favorite, share, report, delete, edit, close, back, fil
- accessible headings/section semantics where React Native/web support it safel
- modal focus/announcement behavior using platform-supported props;
- image alt/accessibility labels that distinguish decorative images from inform
- form errors associated with their field and announced after submit.

Add tests with React Native Testing Library for the shared primitives and at le

10. Add a full loading/error/empty/retry state matrix

Status: Recommended; source contains shared state components but usage is not uniform.

Why: Social apps often fail under offline, slow, permission-denied, and empty conditions. A spinner alone leaves users without recovery or explanation.

What it should look like in the app: Every major screen has skeleton/initial loading, pull-to-refresh, inline retry, meaningful empty copy, mutation errors, and offline/degraded feedback. No blank white screen or infinite spinner.

Cline-ready prompt:

Plain Text

Standardize async UI states for one vertical: the Events list and event detail

Inspect existing LoadingSpinner, Skeleton, EmptyState, ErrorState, React Query

Implement a state matrix:

- initial loading: layout-matching skeleton, not a full-screen spinner when str
- empty: localized explanation plus the most useful next action;
- error: localized non-technical message, retry button, and safe error logging;
- refreshing: preserve existing content while showing refresh progress;
- loading next page: footer indicator;
- mutation pending: disable only the affected action and expose busy state;
- offline/network failure: explain that retry is available and avoid data loss.

Do not duplicate English/French strings inline. Add translation keys in both la

11. Reduce the web bundle and improve startup performance

Status: Confirmed bundle-size concern; exact bottleneck needs profiling.

Why: The exported JS bundle is 6.48 MB and contains 3,826 bundled modules. This can delay LCP and first interaction on slower networks. The performance rule is measure, fix, measure again.

What it should look like in the app: Public/auth pages load without shipping authenticated feature code unnecessarily; heavy media/PDF/video/chart code is loaded only when needed; navigation feels responsive.

Cline-ready prompt:

Plain Text

Profile and reduce Pulse web startup cost without speculative rewrites.

Baseline first:

- run `npx expo export --platform web`;
- record JS/CSS bundle sizes and export warnings;
- use a production-like local server and browser performance trace if available
- identify the largest module groups and route imports.

Then implement only measured wins:

- lazy-load authenticated route groups or heavy rarely-used screens where Expo
- lazy-load PDF/video/media viewers and large editor/filter surfaces;
- remove unused imports and duplicate icon/library paths only when confirmed un

- ensure below-the-fold images use explicit dimensions and appropriate cache/lo
- avoid importing all legal content or all feature data into the initial public
- preserve deep links, auth redirects, and loading fallbacks.

Do not add arbitrary memoization. Do not replace libraries solely by preference

12. Optimize image/video upload and display paths

Status: Recommended; media is a major part of feed/performance risk.

Why: User media can dominate bandwidth, storage, memory, and upload time. The repository already uses `expo-image`, but upload dimensions/format/size policy should be explicit and consistent.

What it should look like in the app: Pickers show constraints before upload, large photos are resized/compressed, thumbnails render first, images reserve layout space, and video uploads fail clearly when unsupported or too large.

Cline-ready prompt:

Plain Text

Create a shared, typed media pipeline for Pulse image uploads first; do not mod

Inspect every `ImagePicker` call, Supabase Storage upload helper, `PostMedia`, prof

Implement:

- a single normalization helper that validates file type and maximum bytes;
- image dimension inspection and resize so uploads have a documented max edge (
- consistent JPEG/WebP quality policy where supported by Expo;
- cancellation handling and user-friendly localized errors;
- stable storage paths and cleanup behavior for replaced images;
- explicit width/height or `aspectRatio` at render sites to prevent layout shift;
- `cachePolicy` and lazy behavior appropriate to avatar, hero, list thumbnail,`
- progress or at least a pending state for uploads.

Preserve original files only where the product explicitly needs them. Add unit

13. Replace broad `select('*')` reads with typed projections

Status: Confirmed pattern across hooks/screens.

Why: Selecting every column increases payload size, couples UI to schema changes, and can accidentally expose fields to client code. Explicit projections improve performance and make data contracts reviewable.

What it should look like in the app: Each screen receives only the fields it renders; generated/shared types document the contract; adding a database column does not silently expand every response.

Cline-ready prompt:

Plain Text

Convert broad Supabase reads to explicit typed projections in one bounded area:

Inspect the database migrations and current TypeScript row types. Define a mini

Requirements:

- no `select('\*')` remains in the targeted files;
- preserve joins and aliases exactly where currently used;
- update TypeScript types so nullable fields remain nullable;
- do not select private or unnecessary fields;
- keep query keys and invalidation behavior unchanged;
- add a test or compile-time check that the card projection contains every rend

Run typecheck, tests, and search the targeted files to verify no wildcard selec

14. Make permissions, privacy, and RLS flows explicit

Status: Recommended security/product hardening; migrations contain many RLS policies and public storage buckets.

Why: Location, photos, messaging, public profiles, invitations, reports, and blocking are sensitive. Permission denial and privacy choices must be clear; database policies need adversarial tests, not just happy-path UI.

What it should look like in the app: Location is optional and explainable, users can see public/private status, blocked users cannot interact through alternate paths, reports are safe, and storage access matches product privacy.

Cline-ready prompt:

Plain Text

Audit Pulse privacy and authorization boundaries without weakening access contr

Inspect all relevant migrations and client flows for profiles, public profiles,

Produce a table of each resource with: read policy, insert/update/delete policy

Pay special attention to:

- private clubs/events and join requests;

- conversation membership and message visibility;
- blocked-user filtering across feed, profiles, search, and messaging;
- public Storage buckets and whether any private media is accidentally public;
- notification insertion permissions;
- invitation token enumeration/expiry/use limits;
- location collection and deletion/retention behavior.

Use migrations for policy changes; never edit production data directly. Add SQL

P2 — Coherence and product-quality improvements

15. Consolidate the design system and remove raw strings

Status: Recommended; the code contains shared Pulse UI components but also many direct `Text` styles and French strings in feature files.

Why: A coherent product needs one spacing/type/color/radius vocabulary and one translation path. Raw strings create language drift and make redesigns expensive.

What it should look like in the app: Buttons, cards, sheets, inputs, headings, chips, and empty states look intentionally related across feed, clubs, events, profile, and messaging; all user-facing copy changes language consistently.

Cline-ready prompt:

Plain Text

Improve Pulse visual and copy coherence in one bounded vertical: the Events list

Inspect the existing design tokens, Button/Card/Text/Badge/Input primitives, gl

Refactor only the Events list:

- replace repeated raw spacing/color/type styles with existing semantic tokens
- use shared primitives for headings, cards, badges, empty/error states, and ac
- move every user-facing string in the targeted files into translation keys in
- ensure date, price, sport, source, and status labels use shared formatters;
- preserve compact/grid/list variants and dark mode.

Do not redesign the entire app. Run `node check\_parity.cjs`, typecheck, tests,

16. Improve onboarding completion and validation resilience

Status: Recommended; onboarding spans five steps, age/legal flows, optional sports/availability, avatar upload, and a signup edge function.

Why: Onboarding is the highest-leverage activation path. It should tolerate interruptions, slow upload, optional fields, invalid dates, duplicate usernames, and server errors without

losing progress or creating partial accounts.

What it should look like in the app: Users can resume safely, see progress, understand required vs optional fields, get field-level errors, and retry a failed final submission without duplicate-account confusion.

Cline-ready prompt:

Plain Text

```
Audit and improve Pulse onboarding reliability without changing its business ru  
Inspect auth/signup routes, SignupStepProgress, Zustand/signup state, utils/val  
  
Implement:  
- explicit required/optional labels and localized field-level errors;  
- date validation that handles invalid calendar dates, timezone boundaries, and  
- safe draft persistence with a clear privacy decision and cleanup after succes  
- retryable avatar upload and final submission states;  
- idempotency protection or duplicate-submit guards for the final action;  
- stable machine-readable server error mapping to UI copy;  
- back/next navigation that does not silently discard valid progress;  
- tests for no-sport, empty availability, underage, duplicate submit, server va  
  
Do not bypass server validation. Keep legal consent explicit. Run all signup te
```

17. Improve search, filtering, and discovery relevance

Status: Recommended.

Why: Pulse has feed search, explore, clubs/events filters, search history, location, and external data. These are powerful but can feel fragmented if filter semantics, reset behavior, ranking, and empty states differ.

What it should look like in the app: One predictable filter sheet, visible active-filter chips, debounced search, recent-search management, distance/date/sport sorting that is explained, and helpful “no results” recovery.

Cline-ready prompt:

Plain Text

```
Audit and standardize search/filter behavior for the Explore screen only.  
  
Inspect app/(tabs)/explore/index.tsx, explore cards, ClubFilters, EventFilters,  
  
Implement a consistent contract:  
- debounce text input without delaying submit unnecessarily;
```

- keep draft filter state separate from applied filter state;
- show active filters as removable chips and provide a clear-all action;
- preserve filters across list/grid view changes and back navigation where appropriate;
- define deterministic sort precedence and document it in code;
- avoid fetching all rows when server-side filters can be applied;
- show result count only when it is accurate;
- add useful empty states with one-tap reset/relax-filter actions;
- keep search history bounded, deduplicated, localized only at display time, and

Add tests for apply/cancel/clear, debounce, deep-link restoration if supported,

18. Strengthen messaging UX and correctness

Status: Recommended; messaging has several large components and recent menu changes, while type errors currently affect conversation actions.

Why: Messaging needs predictable real-time state, delivery/read semantics, keyboard behavior, long-message handling, retry, and safe edit/delete behavior.

What it should look like in the app: Conversations open quickly, new messages appear once, failures can retry, the keyboard does not obscure the composer, menus are accessible, and edit/delete states are clear.

Cline-ready prompt:

Plain Text

Improve Pulse one-to-one conversation reliability after the typecheck is green.

Inspect conversation list/detail screens, useConversations, message hooks/actions

Implement only measured/confirmed issues:

- ensure realtime subscriptions are created once, cleaned up on unmount, and do not
- use stable message IDs and optimistic messages with reconciliation, not duplicates
- expose send failure with retry and preserve unsent draft when safe;
- keep composer above the keyboard on iOS/Android/web;
- define edit/delete time/state rules consistently with server permissions;
- give message action menus accessible labels and proper modal dismissal;
- handle long text, links, images, and empty conversations without overflow;
- paginate older messages and preserve scroll position when prepending.

Add focused tests for subscription cleanup, optimistic failure, duplicate prevention

19. Add offline and poor-network behavior

Status: Recommended.

Why: Mobile sports/social use frequently occurs outdoors or in crowded venues. React Query caching alone is not a complete offline experience.

What it should look like in the app: Cached lists remain readable, a subtle offline banner explains freshness, safe mutations queue or fail clearly, and retry/reconnect behavior is predictable.

Cline-ready prompt:

Plain Text

Add a conservative offline UX to Pulse without silently queueing destructive op

Inspect TanStack Query setup in app/_layout.tsx, auth state, network/platform A

Implement:

- network connectivity detection with a small localized banner;
- sensible stale/cache defaults for read-only feed, clubs, events, and profile
- keep cached content visible while offline;
- disable or clearly explain mutations that cannot be safely queued;
- allow explicit retry after reconnect;
- never queue delete, leave, block, report, or account-security operations auto
- log offline transitions without user data.

Add tests for offline render, reconnect invalidation, and blocked destructive m

20. Add observability and privacy-safe error reporting

Status: Recommended; PostHog is configured and raw console logging is present in app code.

Why: Production failures need context, but logs must not leak emails, message bodies, tokens, location, or health/profile data.

What it should look like in the app/project: Users receive a friendly error ID or retry path; developers can identify route, operation, app version, platform, and correlation ID without collecting sensitive content.

Cline-ready prompt:

Plain Text

Create a privacy-safe error and performance reporting layer for Pulse.

Inspect PostHog setup, app/_layout.tsx, auth, Supabase calls, and all `console.

Implement:

- a typed logger with levels and environment gating;

- a safe error normalizer that records operation, route, app version, platform,
- redaction of emails, access tokens, message/post bodies, coordinates, and full
- consistent user-facing error messages using translation keys;
- optional PostHog exception/performance events only when analytics consent/con
- removal of debug logs such as conversation payload dumps;
- a global error boundary/fallback for unexpected render errors where Expo Router

Do not capture raw user-generated content. Add unit tests for redaction and log

21. Improve navigation, deep links, and back behavior

Status: Recommended.

Why: Expo Router has many nested groups, public/private routes, modal-like routes, and profile/entity deep links. Back behavior and auth redirects are core usability, especially on Android.

What it should look like in the app: A shared link opens the intended entity, authenticated users are redirected back to the original destination, back never loses a form unexpectedly, and modal routes dismiss before leaving the parent flow.

Cline-ready prompt:

Plain Text

Audit Pulse navigation and deep links without changing route names unless requi

Inspect app/_layout.tsx, auth/public/tabs/create/join/profile layouts, Link/Sha

Create a route matrix covering:

- unauthenticated access to protected routes;
- authenticated access to auth routes;
- public club/event/profile links;
- invitation links;
- modal sheets and nested detail pages;
- unsaved create/edit forms;
- notification links.

Implement only verified defects:

- preserve intended destination through auth redirect;
- avoid duplicate stack entries from repeated taps;
- dismiss modal layers before parent navigation;
- add unsaved-change protection only to forms that can actually lose data;
- make share URLs stable and include fallback text;
- add tests for the redirect/deep-link matrix where possible.

Do not introduce a second navigation library or alter server authorization.

22. Improve localization, formatting, and accessibility language quality

Status: Recommended; translation parity passes, but source inspection found many inline French strings and at least one hard-coded notification label in an accessibility string.

Why: Key parity does not guarantee that every visible or announced string is translated, grammatically correct, date/number formatted, or culturally appropriate.

What it should look like in the app: Language switching updates visible copy and accessibility announcements; dates, times, prices, plural counts, and country names follow locale conventions.

Cline-ready prompt:

Plain Text

Perform a localization quality pass on the Notifications screen only.

Inspect the screen, hooks, translations, date/number helpers, and accessibility

Implement:

- move every user-facing and accessibility string into translation keys in FR a
- add plural-aware translations for unread counts and member/comment counts usi
- format dates/times/currency using locale-aware APIs and the user's language/t
- ensure notification type labels have safe fallbacks for unknown server values
- preserve message content as user-authored text while translating surrounding l
- test language switching and zero/one/many values.

Run `node check\_parity.cjs`, typecheck, and tests. Do not edit all 1000 keys or

P3 — Product opportunities after the foundation is healthy

These are valuable possible features, but they should come after P0/P1 work because new features multiply the current quality and performance debt.

Opportunity	Why it may matter	Sensible first version
Personalized activity feed	More relevant content can improve retention.	Rank only from existing follows, sports, location, and recency; keep a chronological fallback and explain “why shown.”
Event reminders/calendar export	Reduces missed attendance.	Opt-in local/push reminder, timezone-safe, calendar deep link, and cancellation when event changes.

Club/event attendance lifecycle	Improves trust and capacity planning.	Going/interested/declined plus capacity/waitlist rules backed by server-side constraints.
Moderation queue	Reports/blocking need operator workflows.	Admin-only review state, audit trail, action reason, and appeal/contact path; never rely on client filtering alone.
Accessibility settings	Supports diverse athletes.	Larger text/layout mode, reduced motion, high contrast validation, and captions/transcripts for media.
Saved searches and alerts	Helps users discover recurring sports/events.	User-controlled saved filter, frequency controls, and unsubscribe path.
Rich media posts	Increases expression.	Enforce file limits, thumbnails, captions/alt text, retryable uploads, and moderation metadata before supporting more formats.
Club roles and permissions	Makes clubs manageable as they grow.	Explicit owner/admin/moderator/member capabilities enforced in RLS and reflected in UI.
Analytics-informed onboarding	Improves activation.	Measure step completion/drop-off with privacy-safe events before changing copy or flow.

Suggested execution order

1. **P0.1–P0.4:** typecheck, tests, quality scripts, dependency alignment.
2. **P0.5–P1.2:** favorite correctness, request multiplication, pagination of the feed.
3. **P1.3–P1.7:** one-screen refactors, accessibility primitives, state matrix, bundle/media profiling.
4. **P1.8–P1.9:** explicit projections and RLS/privacy audit.
5. **P2.1–P2.8:** design coherence, onboarding, discovery, messaging, offline, observability, navigation, localization.
6. **P3:** feature experiments backed by analytics and server-side authorization.

For a small team using inexpensive IDE agents, create one branch or PR per prompt, keep each change under roughly 300–500 lines when possible, and require the agent to report validation commands. Never ask an agent to “refactor the whole app” in one pass.

Reusable low-cost-agent operating prompt

Use this prefix with every task prompt above:

Plain Text

You are a careful implementation agent in the Pulse Expo/React Native repository.

Rules:

- Read the named files and nearby types before editing.
- Search for existing helpers/primitives before creating new ones.
- Make one bounded change; do not opportunistically refactor unrelated code.
- Preserve existing route names, database contracts, translations, dark mode, a
- Never use ``any``, ``@ts-ignore``, ``@ts-expect-error``, unsafe casts, or swallowed
- Do not run destructive database commands or production operations.
- Do not expose secrets or log user content.
- After editing, run the exact validation commands requested by the task.
- If validation fails for a pre-existing reason, show the error and distinguish
- Finish with: summary, changed files, tests/checks run, remaining risks, and a

Final acceptance checklist

Before merging any improvement, verify:

- `npx tsc --noEmit` passes.
- `npm test -- --runInBand` passes.
- `npm run lint` is installed, configured, and passes.
- `npx expo export --platform web` passes.
- `npx expo install --check` is clean or documented with an explicit exception.
- iOS and Android are checked for any native or keyboard/permission change.
- FR/EN parity passes and newly visible strings are translated.
- Keyboard navigation/VoiceOver/TalkBack behavior is checked for interactive changes.
- Mobile widths around 375 px and tablet/desktop widths are checked for layout changes.
- Supabase queries are bounded, explicit, authorized, and do not introduce per-card N+1 behavior.

- Optimistic mutations have rollback and duplicate-tap protection.
- Logs and analytics contain no secrets or user-generated sensitive content.
- Performance changes include before/after measurements rather than assumptions.

Note on requested skills

The review followed the relevant web-design-reviewer and performance-optimization principles: inspect source structure, measure the build, distinguish evidence from recommendations, and avoid premature optimization. The skill-creator instructions are not applicable to the repository changes because this task requested an audit and implementation prompts, not creation or modification of a reusable Manus skill.

Repository snapshot

This report is delivered as a new Markdown artifact and does not modify application source code.

End of audit.